

A Graph-Based Control Interface for Traffic Signals on Heterogeneous Road Networks

Bertil Braun

Abstract

Traffic signals do not choose from one universal action set: the legal ways of moving through a junction depend on its geometry, lane connectivity, and signal design. This report describes a movement-centric interface that separates representation, learned prioritization, and local action construction. The model groups directed road corridors into LANEGROUP nodes and represents each legal signal-controlled path through a junction as a MOVEMENT node. A shared typed message-passing network produces one scalar score per MOVEMENT. At each junction, a fixed incidence matrix aggregates those scores into logits for phases constructed automatically from SUMO controlled links and conflict data. The learned parameter shapes therefore do not depend on network size or local phase count. PPO experiments support the implementation: policies transfer across synthetic grid sizes and aspect ratios, execute on five heterogeneous city scenarios, and expose a clear limitation when signal coverage changes outside the training distribution. The result is a practical representation and control pipeline, not a claim of a new reinforcement-learning algorithm or universal traffic-control generalization.

1 Motivation

A traffic signal determines which vehicle paths through an intersection may proceed together. The number and meaning of those choices vary: a three-arm junction, a regular four-arm junction, and an intersection with dedicated turn lanes need not share an action schema. Consequently, phase index 2 at one controller has no intrinsic relationship to phase index 2 at another. A policy whose output weights correspond directly to fixed phase indices embeds assumptions about one predefined junction layout.

This implementation instead uses a MOVEMENT—a legal controlled turn from an incoming corridor to an outgoing corridor—as the interface between shared learning and local signal operation. Movements retain common semantics when their number and compatible combinations change. The learned model can therefore emit a variable number of MOVEMENT scores, after which each junction maps its own scores to its own phase set.

The report contributes an integrated implementation:

1. a city-level graph of directed LANEGROUP and MOVEMENT nodes;
2. typed communication for upstream demand, downstream supply, and unsignalized pass-through context;
3. a shared two-block message-passing policy with a scalar MOVEMENT head;
4. automatic construction of junction-specific, conflict-derived phase sets; and
5. an OSM-to-SUMO, PPO-training, and evaluation workflow.

Movement pressure and movement-structured control have substantial prior lineage. Max-pressure methods score compatible stages from constituent traffic flows [1]; PressLight connects pressure to learned control [2]; and FRAP structures a policy around movements and phase competition [3]. TransferLight also maps movement representations to variable phase spaces, using a learned hierarchical phase encoder [4]. The present report does not claim to originate movement scoring. Its focus is the transparent combination of a city-level typed graph, a scalar learned interface, and phases generated by the network-construction pipeline.

2 Traffic-signal concepts

2.1 What the controller chooses

Consider a vehicle approaching a four-arm intersection from the south. Depending on the lane connections, it may continue straight, turn left, or turn right. Each such input-to-output path is a *traffic movement*. A movement is therefore more specific than “the north–south road”: it identifies where traffic enters the junction and where it leaves. Straight travel is a movement too; the word does not imply that the vehicle must physically turn.

A signal cannot normally give every movement green at once because some paths cross or merge. Instead, it selects a *phase*: a set of movements that the signal plan allows to receive green together. For example, the two opposing straight movements on a north–south road may form one phase, while east–west traffic is held red. A later phase might serve the east–west movements or a protected left turn. Here, “phase” always means one selectable green combination, not a training stage or a time step.

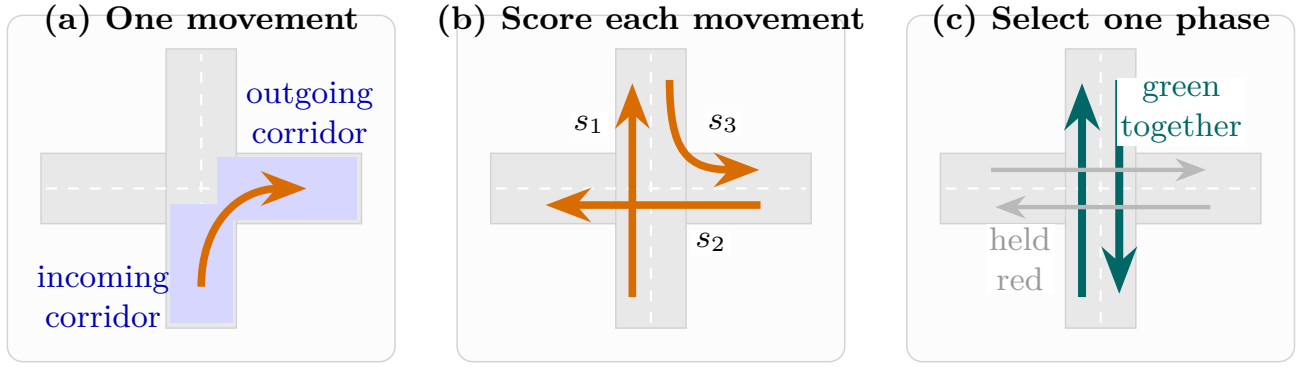


Figure 1: The control vocabulary used throughout the report. A movement is one legal path from an incoming corridor to an outgoing corridor. The learned model scores movements individually. A phase is a junction-specific compatible set that may receive green together; the drawing shows one simple example rather than a universal phase template.

The controller’s action at one decision is the choice of one available phase. It does not independently flip every lamp, nor does it generate an arbitrary red–yellow–green string. The network-building step determines the legal green combinations, and the runtime inserts the required yellow transition and enforces minimum green. This separation is important: learning ranks valid choices, while deterministic code handles signal compatibility and timing rules.

2.2 From physical lanes to model objects

The road network contains individual physical lanes, but treating every lane segment as an independent graph node would preserve detail that is often irrelevant to the policy while making long corridors unnecessarily deep. The implementation therefore uses a `LANEGROUP` as its corridor-level state object. A `LANEGROUP` is directed: the two travel directions of the same street are separate objects because their queues, speeds, and destinations differ. It may span an unambiguous continuation through an unsignalized junction, but it stops before a branch where different destinations must remain distinguishable.

At a controlled junction, the legal connection from an incoming `LANEGROUP` to an outgoing `LANEGROUP` becomes a `MOVEMENT` node. The capitalized terms refer to these model objects; lowercase “lane”, “movement”, and “phase” refer to the corresponding traffic concepts. The junction itself owns its local `Movements` and `phases` but is not a GNN node. This produces three deliberately different roles:

Table 1: Plain-language interpretation of the representation.

Object	What it represents	What it contributes
<code>LANEGROUP</code>	One direction of a road corridor	Queue, speed, occupancy, capacity, arrivals, and downstream space
<code>MOVEMENT</code>	One legal input-to-output path through a controlled junction	Turn-local demand, turn type, controlled links, and recent green state
<code>Phase</code>	A compatible local set of <code>Movements</code>	One action available to that junction

The distinction lets the model ask two separate questions. The graph asks what traffic conditions surround each possible movement. The local action mapping then asks which compatible collection of those movements should receive green.

3 Representation and control interface

3.1 One decision from observation to signal action

Before describing the individual components, the complete online path is useful:

1. SUMO and the detector logic update the features of every `LANEGROUP` and `MOVEMENT` in the city graph.
2. The shared GNN exchanges information along typed edges and produces one real-valued score for every `MOVEMENT`.
3. Each junction gathers the scores of its own `Movements`.
4. For every locally valid phase, it sums the scores of the `Movements` that phase would activate.
5. A runtime mask removes choices that are temporarily unavailable, for example because minimum green has not elapsed.
6. The policy samples one remaining phase; when it differs from the current phase, the runtime executes yellow before applying the new green.

Only step 2 is the learned graph representation and scorer. The phase definitions, score-to-phase reduction, mask, and transition execution are inspectable operations. This is how one parameter set can face a different graph

size and a different number of actions at each intersection without allocating a learned output neuron to every possible local phase.

3.2 From road corridors to a typed graph

A **LANEGROUP** is a directed corridor between controller-relevant endpoints. Opposite travel directions are distinct nodes because they have different queues, speeds, capacity, and destinations. Unambiguous continuations through unsignalized junctions may be contracted; contraction stops before a branch would create false reachability. Dynamic features summarize the detector region near the downstream end, including vehicle and queue counts, occupancy, speed, storage, arrival and departure rates, and queue-tail arrival estimates. Static features describe corridor scale.

A **MOVEMENT** joins one input **LANEGROUP** to one output **LANEGROUP** through a controlled junction, corresponding to the vehicle path introduced in Figure 1. It holds turn-local features such as demand, turn class, controlled-link count, and recent green state. Junctions own Movements and phases but are not graph nodes.

Every **MOVEMENT** has four directed relation types:

$$L_{\text{in}} \rightarrow M, \quad L_{\text{out}} \rightarrow M, \quad M \rightarrow L_{\text{in}}, \quad M \rightarrow L_{\text{out}}.$$

Message direction is information flow rather than vehicle flow. In particular, $L_{\text{out}} \rightarrow M$ returns downstream storage and spillback context to a turn that would feed that corridor. This permits a movement to be unattractive not only when few vehicles want to use it, but also when the road beyond the junction is already full. At an unsignalized junction, legal pass-through connections become weighted **LANEGROUP**-to-**LANEGROUP** edges instead of controllable **MOVEMENT** nodes. Their weight decays with connector free-flow time t_{ff} :

$$w = \exp(-t_{\text{ff}}/30 \text{ s}).$$

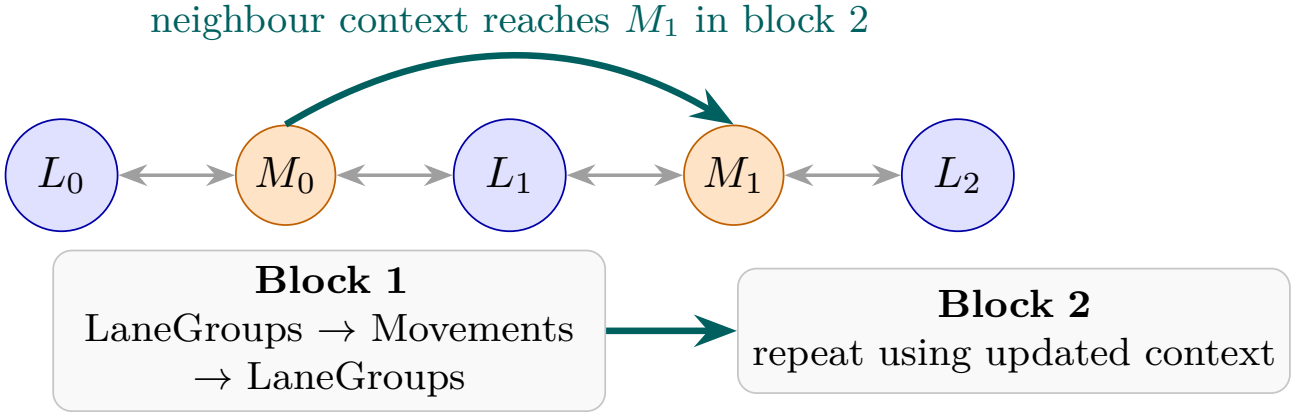


Figure 2: The typed bipartite representation and why the second update block matters. The first block updates Movements from endpoint LaneGroups and returns them to LaneGroups. In the second block, a Movement can use context deposited by another Movement through a shared corridor.

3.3 Two update blocks, not graph attention

The active model is a typed mean-aggregation graph neural network, not a graph attention network. LaneGroups and Movements are embedded separately. Intuitively, each relation type answers a different question. Incoming-LaneGroup messages describe demand approaching a Movement; outgoing-LaneGroup messages describe the space into which it would discharge. Messages in the reverse direction deposit that movement context back onto the adjoining corridors. The transformations are shared across every object of the same type, which is the source of the model’s size independence.

For update block k , the essential computation is

$$M^{k+1} = \phi_M(M^k, \text{mean}_{\text{in}}(L^k), \text{mean}_{\text{out}}(L^k)), \quad (1)$$

$$L^{k+1} = \phi_L(L^k, \text{mean}_{M \rightarrow \text{in}}(M^{k+1}) + \text{mean}_{\text{conn}}(L^k), \text{mean}_{M \rightarrow \text{out}}(M^{k+1})). \quad (2)$$

Each relation has its own linear transformation; neighbours are averaged rather than assigned learned attention weights.

Two blocks are used. The distinction is important. Suppose Movement M_0 releases traffic into corridor L_1 , and a downstream Movement M_1 controls traffic leaving that corridor. The first block lets M_0 write information onto L_1 . Only the next block lets M_1 read that updated corridor state. Thus the second block is the first opportunity for one Movement to affect another through a shared **LANEGROUP** (Figure 2). The depth is configurable, but this report neither assumes that more blocks are always beneficial nor presents an optimal-depth claim. The final Movement embeddings are mapped to scalar scores s_m . The critic mean-pools the Movements owned by a junction and estimates one value per controller.

3.4 Automatic phases and local action mapping

OSM tags do not directly define the action space used by the policy. They identify roads, lanes, and signalized locations, but the controller still needs a concrete list of compatible green choices like the one illustrated in Figure 1c. The build pipeline uses SUMO `netconvert` [5] to obtain lane connections, controlled-link indices, request indices, and request-conflict (`foes`) data. Links that must activate together are first combined into atomic groups. An atomic group is rejected if it contains an internal conflict. Two groups are incompatible when SUMO reports a conflict in either direction or when different incoming approaches merge into the same outgoing edge.

The compatible groups form a graph. Bron–Kerbosch enumeration [6] returns every maximal clique, and each maximal compatible set becomes one selectable green phase. “Maximal” means that no additional compatible movement group can be added to that particular set. It does not mean “the phase containing the largest number of movements”. A smaller protected-turn set therefore remains an action when it cannot safely be enlarged, even if another phase contains more Movements. This is conflict-derived construction under the implemented SUMO and merge rules, not formal verification of a real signal plan.

For junction j , let $A_j \in \{0, 1\}^{|P_j| \times |M_j|}$ record which local Movements are green in each selectable phase. The shared GNN produces $\mathbf{s}_j$, and phase logits are simply

$$\ell_j = A_j \mathbf{s}_j, \quad \ell_{j,p} = \sum_m A_{j,p,m} s_m.$$

For a concrete example, if one phase activates Movements with scores 1.2, 0.7, and -0.4 , its logit is 1.5. Another junction may have two, six, or twelve phases; only its local matrix A_j changes. The learned scorer and its parameter shapes remain the same. An availability mask removes phases that cannot be chosen at the current decision. The remaining logits define a local categorical policy. This fixed sum is an explicit, inspectable interface and inductive bias; it is not claimed to be the only or optimal phase aggregation.

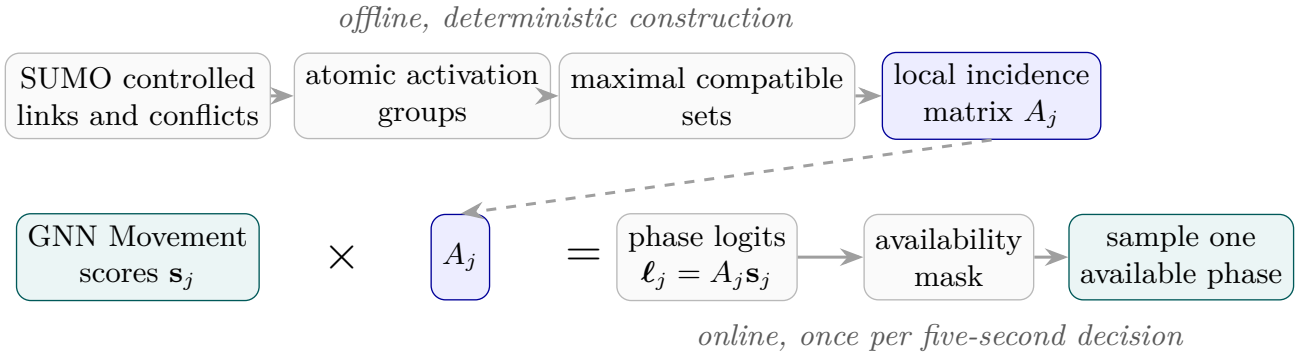


Figure 3: Offline phase construction is separate from online learning. The GNN supplies a variable-length Movement-score vector; a junction-local incidence matrix and temporary availability mask produce the action distribution.

3.5 A miniature worked decision

Consider a small junction with four controlled Movements. Movements m_1 and m_2 are the two opposing straight paths and may receive green together. Movements m_3 and m_4 form a second compatible choice. For this example, the automatically constructed phase set is $P_j = \{p_1, p_2\}$, with

$$A_j = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}.$$

The rows are phases and the columns are Movements. The first row says that phase p_1 activates m_1 and m_2 ; the second activates m_3 and m_4 . The matrix is stored with this junction and contains no learned parameters.

Suppose the GNN reads the current city state and returns

$$\mathbf{s}_j = [1.2 \quad 0.7 \quad 0.6 \quad -0.4]^\top.$$

The resulting phase logits are

$$\ell_j = A_j \mathbf{s}_j = [1.9 \quad 0.2]^\top.$$

The policy therefore prefers p_1 , because its two constituent Movement scores sum to the larger value, but this is not yet a light command. If both phases are currently available, the logits define the two-action categorical distribution used by PPO. If a minimum-green rule temporarily forbids switching to p_2 , the mask removes p_2 before sampling. After a phase is selected, the runtime decides whether a yellow transition is necessary.

A different junction may supply a 6×11 incidence matrix instead of the 2×4 matrix above. The GNN still performs the same typed updates and produces one score per Movement. Only the fixed local reduction changes, which is the central mechanism behind variable intersection geometry and variable action counts.

4 Training and evaluation

PPO [7] is used as an optimizer and is not re-derived here. The anchored controller makes a decision every 5s, starts an immediate 3s yellow when switching, and requires one decision of minimum green. Training uses two message-passing blocks, four PPO epochs, 200 decisions per rollout, and entropy coefficient 0.001. Its local reward combines vehicle progress, discharge, braking, and gridlock terms with weights 1/10/10/0.02; global, throughput, flow, and direct switching terms are zero.

Mixed graph sizes require one practical correction: equal rollout counts are not equal PPO weight because larger graphs yield more junction actions per transition. Rollouts are therefore allocated to approximately balance the number of local action samples contributed by each training shape or city.

Three evaluations answer progressively less controlled questions:

- **Grid size and shape.** Three independently trained mixed-grid policies saw square and rectangular grids with longest axis at most five. They were evaluated without further learning on ten geometries, including 6×6 , using six fresh traffic seeds.
- **Signal coverage.** Removing controllers tests whether dense-signal training transfers to graphs where unsignalized pass-through edges replace controlled Movements. A separate 50%-coverage training run tests whether exposure to those gaps changes the outcome.
- **Cities.** One policy was trained with rollouts from Karlsruhe, Mannheim, Heidelberg, and Freiburg and evaluated on those scenarios plus Stuttgart. All runs use the fixed terminal checkpoint at iteration 60; training duration was not selected by stopping on an evaluation peak.

Sampled execution is primary because it matches the policy optimized by PPO. Greedy execution and full per-seed results are retained in the appendix. Max pressure is the main traffic-control baseline, with queue, fixed-time, and uniform-random policies as additional references.

5 Results

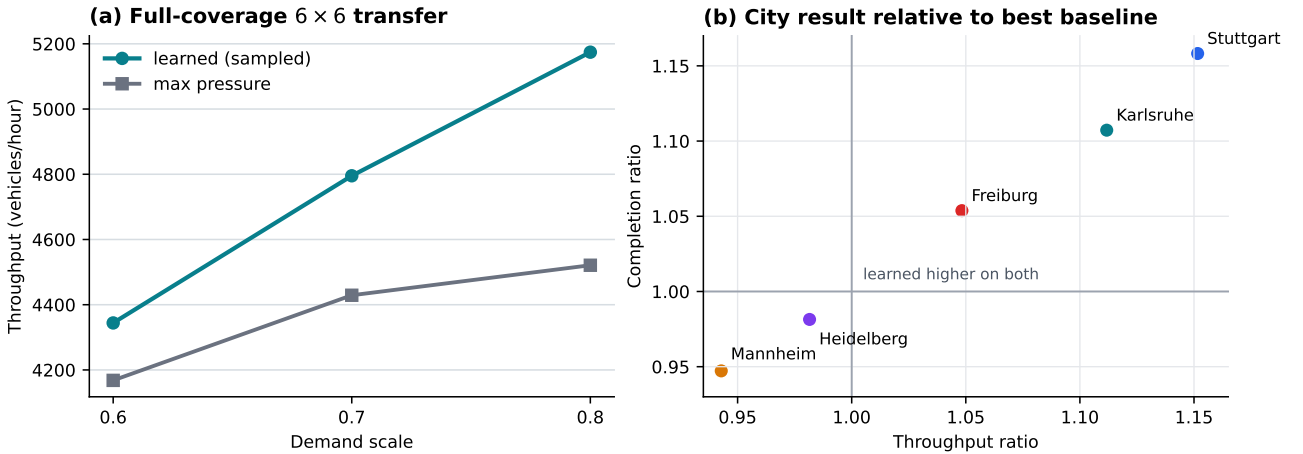


Figure 4: Succinct evaluation summary. Left: the sampled mixed-grid policy transfers to the larger full-coverage 6×6 grid across all evaluated demands (three training seeds and six fresh traffic seeds). Right: city throughput and completion relative to the strongest baseline for each metric, showing both successful and mixed cases.

Synthetic topology transfer. At full signal coverage, the learned policy retained useful behaviour as grid size and aspect ratio changed. On 6×6 , sampled throughput rose from 4344 to 5174 vehicles/hour across demand scales 0.6 to 0.8, compared with 4168 to 4521 for max pressure. Completion improved by 3.5, 6.6, and 10.6 percentage points at the three demands. Transposed $5 \times 2/2 \times 5$ and $5 \times 3/3 \times 5$ pairs were also similar. These results support reuse across sizes and shapes generated by the same synthetic process; they do not establish transfer to arbitrary topology.

Coverage is distribution-sensitive. A policy trained only on dense signals deteriorated sharply when half or more eligible controllers were removed. Training on several 50%-coverage layouts recovered throughput and completion near max pressure on fresh 25% and 50% layouts, but did not yield uniformly good waiting density or higher-coverage behaviour. The graph can carry information through unsignalized gaps, but the policy must encounter the changed control distribution during training.

City behaviour is heterogeneous. The city graphs range from 286 to 708 LaneGroups, 271 to 508 Movements, and 41 to 84 policy-controlled junctions. One parameter set runs on all five. Sampled control is clearly positive in Karlsruhe and improves Stuttgart throughput/completion over every baseline. It is close to fixed time in Heidelberg, trails queue control in Mannheim, and improves Freiburg throughput and completion while not improving wait density. The five-city mean is 3345.6 vehicles/hour and 64.93% completion, versus 3036.8 and

59.37% for max pressure. The city experiment is one training run and should be read as a feasibility study, not a general city-transfer estimate.

6 Limitations and conclusion

Table 2: Intended interpretation of the evaluation evidence.

Evaluation	Supports	Does not establish
Synthetic grids	Size/shape reuse within the grid generator	Transfer to arbitrary topology
Signal coverage	Diagnosis of a control-distribution shift	Coverage-invariant control
Five cities	Execution feasibility and heterogeneous outcomes	A general city-transfer estimate

6.1 What the architecture provides

The main architectural benefit is not that a graph model automatically solves traffic control. It is that the learned computation is attached to objects with reusable meaning. A `LANEGROUP` always describes traffic travelling in one direction along a corridor, and a `MOVEMENT` always describes one possible passage through a controlled junction. The same relation-specific transformations can therefore be applied whether a network contains tens or hundreds of junctions. Unlike a fixed phase-index output layer, the learned parameters do not have to reserve a slot for “phase 3” and hope that phase 3 means the same thing elsewhere.

The Movement score also creates a narrow boundary between learning and signal logic. The GNN can combine approaching demand, downstream capacity, and neighbouring Movement context, but it never has to invent a legal light configuration. Conversely, phase construction and runtime timing do not have to understand the neural embedding; they only consume scalar preferences. This makes the action path easier to inspect and permits a newly built network to supply its own phase-incidence matrices while reusing the scorer.

Two update blocks extend the controller beyond an isolated intersection without creating a global centralized state vector. Information can pass from one Movement into a shared corridor and then into a neighbouring Movement. The receptive field remains local, and the same operation is repeated throughout the graph. This is the practical reason for using a GNN here: variable-sized relational computation and parameter sharing, rather than attention as a mechanism in itself.

6.2 Boundaries

The architecture removes fixed graph and phase dimensions, but structural compatibility is not distributional robustness. Sparse signal coverage, different demand, and irregular city traffic can still change what the same local observations mean. Detector features are local, the phase sum assumes additive Movement utilities, and the conflict rules inherit the fidelity of the imported SUMO network. Sampled and greedy execution also differ materially, so deterministic argmax control cannot be substituted silently for the trained policy.

6.3 Conclusion

Within those boundaries, the implementation provides a compact way to connect a shared graph policy to heterogeneous traffic networks. LaneGroups give traffic state a directed corridor representation; Movements provide reusable controlled primitives; the second message-passing block introduces neighbouring Movement context; and deterministic phase construction keeps local compatibility outside the learned model. The synthetic experiments give the clearest evidence for size and shape reuse, while coverage and city results identify where training distribution remains decisive.

References

- [1] Pravin Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195, 2013.
- [2] Hua Wei, Chacha Chen, Guanjie Zheng, Kan Wu, Vikash Gayah, Kai Xu, and Zhenhui Li. PressLight: Learning max pressure control to coordinate traffic signals in arterial network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1290–1298, 2019.
- [3] Guanjie Zheng, Yuanhao Xiong, Xinshi Zang, Jie Feng, Hua Wei, Huichu Zhang, Yong Li, Kai Xu, and Zhenhui Li. Learning phase competition for traffic signal control. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, pages 1963–1972, 2019.
- [4] Johann Schmidt, Frank Dreyer, Sayed Abid Hashimi, and Sebastian Stober. TransferLight: Zero-shot traffic signal control on any road-network. *arXiv preprint arXiv:2412.09719*, 2024.
- [5] Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. Microscopic traffic simulation using SUMO. In *2018 21st International Conference on Intelligent Transportation Systems*, pages 2575–2582, 2018.
- [6] Coen Bron and Joep Kerbosch. Algorithm 457: Finding all cliques of an undirected graph. *Communications of the ACM*, 16(9):575–577, 1973.
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

A Implementation details

Table 3: Anchored training and control settings.

Setting	Value
Decision interval	5 s
Yellow transition	immediate, 3 s
Minimum green	one decision
Message-passing update blocks	2
Hidden dimension	64
Decisions per rollout	200
PPO epochs per update	4
Entropy coefficient	0.001
Progress/discharge/braking/gridlock	1/10/10/0.02
Global/flow/throughput/switch reward	0

Table 4: Structural diversity of the city scenarios. Phase range is the minimum–maximum number of selectable phases per policy-controlled junction.

City	Controllers	LaneGroups	Movements	Phase range
Karlsruhe	41	286	271	2–10
Mannheim	84	708	467	2–7
Stuttgart	69	519	508	2–12
Heidelberg	56	408	365	2–10
Freiburg	58	425	416	2–11

B Detailed evaluation

Table 5: Full-coverage 6×6 result. Learned values average three training seeds and six traffic seeds; max pressure averages the six traffic seeds.

Demand	Policy	Throughput/h	Completion	Wait density
0.6	learned sampled	4344	87.2%	0.077
	max pressure	4168	83.6%	0.082
0.7	learned sampled	4795	84.8%	0.114
	max pressure	4429	78.2%	0.119
0.8	learned sampled	5174	82.2%	0.175
	max pressure	4521	71.5%	0.131

Table 6: Iteration-60 city means over three traffic seeds. The baseline shown is the policy with the highest throughput in that city; wait density is reported in seconds per metre.

City	Policy	Throughput/h	Completion	Wait density
Karlsruhe	learned sampled	3452	93.86%	0.075
	max pressure	3105	84.77%	0.243
Mannheim	learned sampled	3200	53.78%	0.796
	queue	3394	56.78%	1.019
Stuttgart	learned sampled	4216	50.93%	0.907
	fixed time	3661	43.97%	0.741
Heidelberg	learned sampled	3190	77.69%	0.181
	fixed time	3250	79.16%	0.127
Freiburg	learned sampled	2670	48.39%	1.108
	fixed time	2547	45.92%	0.934

Sampled versus greedy. Greedy city control averaged 3182.6 vehicles/hour, 61.89% completion, and 0.980 s/m wait density, compared with 3345.6, 64.93%, and 0.613 s/m for sampled control. The gap motivates using the sampled policy as the primary result.

Coverage study. Dense-signal training transferred poorly to reduced coverage. A separate policy trained on five 50%-coverage 4×4 layouts matched max-pressure throughput and completion closely on fresh 25% and 50% layouts under sampled execution. At 75%, sampled throughput and completion were lower; greedy control retained throughput but accumulated more wait. This one-seed study diagnoses a training-distribution effect rather than establishing coverage-invariant control.

Evaluation units. Grid means should distinguish three independent policy-training seeds from the six traffic seeds used to evaluate each policy. The city study contains one policy-training seed and three traffic seeds. No claim in the main text relies on treating all traffic episodes as independent policy replications.